

WORK BREAKDOWN STRUCTURE (WBS)

Feature Flag & Dynamic Config Manager | Problem Statement #43

1. Project Objective

Break the proposed Feature Flag & Dynamic Config Manager project into manageable planning, design, implementation, integration, verification, and delivery work packages. This WBS is a proposed plan; it does not claim that implementation or testing has already been completed.

2. WBS Hierarchy

WBS ID	Work Package	Description
1.0	Project Management & Planning	Coordinate scope, schedule, responsibilities, and progress.
1.1	Review problem statement and handout	Confirm required features, stakeholders, deliverables, and constraints.
1.2	Define project scope and assumptions	Record in-scope capabilities and open design decisions.
1.3	Plan tasks and milestones	Sequence work packages and track completion.
2.0	Requirements Engineering	Document and validate what the system must do.
2.1	Prepare functional requirements	Specify FR-001 through FR-005 with acceptance criteria.
2.2	Prepare non-functional requirements	Specify NFR-001 and NFR-002 with verification criteria.
2.3	Prepare Requirements Traceability Matrix	Map each requirement to use cases and planned verification.
2.4	Review requirements	Check clarity, consistency, testability, and relevance.
3.0	System Analysis & Design	Describe system behavior and its proposed structure.
3.1	Identify actors and use cases	Define Software Engineer, Release Manager, and Client SDK interactions.
3.2	Create UML use-case diagram	Draw system boundary, use cases, associations, include and extend.
3.3	Write use-case flow specification	Document UC-03 preconditions, postconditions, success and alternate flow.
3.4	Prepare Software Requirements Specification	Describe purpose, scope, users, interfaces, constraints, and requirements.
4.0	Implementation Planning & Development	Plan and build the platform capabilities if implementation is in scope.
4.1	Select architecture and technology	Decide components, storage, API, authentication, and SDK update approach.
4.2	Design flag data model	Define flag keys, states, targeting rules, cohorts, and environments.
4.3	Develop flag management functions	Create, view, update, and deactivate flags.

4.4	Develop evaluation and targeting	Evaluate user context and return boolean flag states.
4.5	Develop rollout and environment controls	Configure percentage rollout and environment-specific overrides.
4.6	Develop SDK synchronization	Propagate configuration changes to connected clients.
5.0	Verification & Quality Assurance	Check requirements and system behavior against acceptance criteria.
5.1	Prepare test cases	Create tests linked to the RTM requirements.
5.2	Test flag management and unique keys	Verify CRUD behavior and duplicate-key handling.
5.3	Test targeting and percentage rollout	Verify cohort assignment and expected flag results.
5.4	Test environment isolation	Verify settings in one environment do not alter another.
5.5	Test SDK synchronization	Verify updated values propagate within the defined interval.
5.6	Benchmark performance and check access control	Measure evaluation latency and verify authorization rules.
5.7	Record defects and retest fixes	Document failures, patches, and retest outcomes.
6.0	Documentation & Submission	Package the required individual project artifacts.
6.1	Prepare README	Describe project and contents of the six submission folders.
6.2	Organize requirements and design artifacts	Store workbook, RTM, diagram, flow, SRS, and WBS.
6.3	Capture project creation evidence	Add genuine GitHub and Jira screenshots.
6.4	Capture Copilot evidence	Add generated-code screenshot or relevant repository link if used.
6.5	Prepare testing evidence	Add test details, results, patch summary, and repository link when available.
6.6	Review and submit repository	Check filenames, links, readability, and push final changes.

3. Key Deliverables

- Requirements workbook containing five FRs, two NFRs, and the RTM.
- UML use-case diagram (editable source and PDF export).
- UC-03 use-case flow specification (DOCX and PDF export).
- Software Requirements Specification (DOCX and PDF export).
- This Work Breakdown Structure (DOCX and PDF export).
- README and genuine project-creation evidence.
- Copilot evidence, if applicable, and testing artifacts when the assigned testing activity is available.

4. Dependencies & Notes

- Requirements and use cases should be reviewed before finalizing the detailed design.
- The architecture and technology choices should be made before implementation work packages are started.

- Verification criteria depend on finalized design details, including the benchmark conditions for NFR-001 and the SDK synchronization interval for FR-005.
- The separate software-testing-tools exercise depends on receiving the game application repository and its four test cases from the instructor; keep this item pending until provided.
- Mark work packages complete only after the corresponding artifact or activity has actually been completed.